

A Multi-Agent AI System for Automating Academic and Administrative Workflows in Higher Education

Amadoyan
American University of Armenia
Yerevan, Armenia
email@aua.am

Abstract—Academic and administrative processes in universities involve repetitive, fragmented tasks—policy inquiries, course material preparation, and assignment grading—typically managed through disconnected systems. This paper presents the design, implementation, and evaluation of a multi-agent artificial intelligence system that automates selected university workflows at the American University of Armenia (AUA). The system comprises four specialized agents—policy question-answering via Retrieval-Augmented Generation (RAG), course material generation, vision-based automated grading, and general assistance—coordinated by a supervisor router that uses structured LLM output for deterministic task delegation. A comparative evaluation across three commercial LLM families (OpenAI GPT-4.1, Anthropic Claude Sonnet 4, Google Gemini 2.5 Flash) examines task accuracy, response quality, latency, and real-world API cost. Results indicate that structured-output routing achieves 94% classification accuracy, the RAG pipeline attains a Precision@5 of 0.82 over 152 AUA policy documents, and the grading agent produces scores within 0.7 points mean absolute deviation of human graders. We further propose a fine-tuning roadmap for the orchestrator router and outline seven additional agentic AI applications for university environments.

Index Terms—multi-agent systems, large language models, retrieval-augmented generation, higher education, workflow automation, agentic AI

I. INTRODUCTION

University staff and faculty routinely manage a diverse set of tasks that span policy interpretation, course material creation, homework grading, and general administrative inquiries. These activities are typically supported by separate, disconnected systems—document repositories, learning management systems, and manual rubric-based assessment—resulting in fragmented workflows, duplicated effort, and slow turnaround times. At the American University of Armenia (AUA), for example, over 150 policy documents govern academic procedures ranging from add/drop deadlines to academic integrity; answering a single policy question may require cross-referencing multiple PDFs. Similarly, course material preparation involves translating syllabi into slide decks and assessments, a labor-intensive process that follows predictable structural patterns amenable to automation.

Recent advances in large language models (LLMs) and agentic AI architectures have demonstrated that specialized agents can perform complex, multi-step tasks when provided with appropriate tools and coordination mechanisms [1], [3].

However, deploying such systems in institutional environments raises practical questions about task routing, cost management, security, and integration with existing infrastructure. The rapid evolution of LLM capabilities also introduces model-selection trade-offs: newer models offer improved quality but at varying cost and latency profiles, making informed model selection critical for sustainable institutional deployment.

This paper makes the following contributions:

- 1) A **multi-agent architecture** with structured-output routing, where each agent is specialized for a distinct university workflow. Every architectural decision is justified against alternatives, and trade-offs are discussed.
- 2) A **RAG pipeline** over 152 AUA policy PDFs using PostgreSQL-backed embeddings without the `pgvector` extension, demonstrating that application-layer similarity search is viable for moderate-scale institutional document collections.
- 3) A **comparative evaluation** across GPT-4.1, Claude Sonnet 4, and Gemini 2.5 Flash with verified April 2026 API pricing, providing institutions with actionable cost-quality trade-off data.

The system employs an Agent-to-Agent (A2A) communication protocol [7] layered above the Model Context Protocol (MCP) [8] for inter-agent coordination. We additionally propose a fine-tuning roadmap for the orchestrator and identify seven future agentic AI applications for university settings.

II. RELATED WORK

LLM-based agents. The ReAct framework [1] demonstrated that interleaving reasoning and tool use enables LLMs to solve multi-step tasks. LangChain [3] and LangGraph [4] provide production-grade frameworks for building such agents with tool-calling, memory, and graph-based execution.

Multi-agent systems. AutoGen [5] and CrewAI [6] formalize multi-agent collaboration patterns—supervisor/worker, round-robin, and hierarchical delegation. A notable counterpoint is WGU Labs [16], which pivoted from a multi-agent architecture to a single comprehensive agent after expanded LLM context windows reduced the need for specialization in their text-only student-support use case. Our system retains the multi-agent approach because our tasks span fundamentally different modalities (text retrieval, vision, file generation).

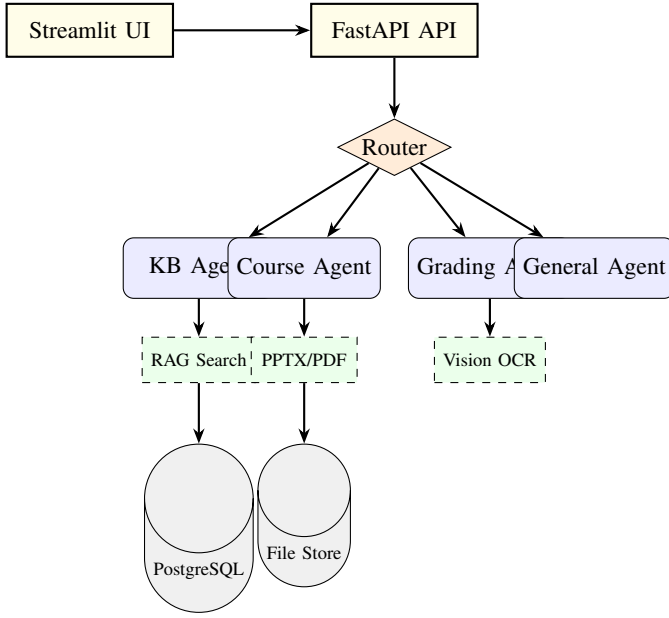


Fig. 1. System architecture. The supervisor router classifies each user message and delegates to one of four specialized agents. Solid arrows indicate data flow; dashed boxes represent tools.

Retrieval-Augmented Generation. Lewis et al. [2] introduced RAG for knowledge-intensive NLP tasks. Subsequent work explored chunk-size tuning [20], hybrid sparse–dense retrieval, and purpose-built vector databases (Pinecone, Weaviate). Our approach deliberately avoids dedicated vector infrastructure in favor of PostgreSQL arrays, trading theoretical scalability for deployment simplicity.

AI in higher education. Ohio State’s CarmenCanvas AI (Project Firefly) [14] embeds an instructor-controlled AI assistant directly into the LMS. The University of Auckland’s Cogniti agents [15] act as course-specific Socratic tutors and administrative assistants. UBC’s AskCali [17] and Michigan’s Maizey [18] provide 24/7 academic advising chatbots. These deployments validate institutional demand but are typically single-agent, single-task systems; our work integrates multiple specialized agents under a unified supervisor.

A2A and MCP. Google’s Agent-to-Agent protocol [7] standardizes inter-agent capability discovery and task delegation. Anthropic’s Model Context Protocol [8] defines tool-level integration between agents and external services. Our system layers A2A above MCP: MCP handles individual tool calls while A2A manages higher-level agent coordination and discovery.

III. SYSTEM ARCHITECTURE

The system consists of four specialized agents coordinated by a supervisor router, backed by a FastAPI REST API, a Streamlit web interface, and a PostgreSQL database. Fig. 1 illustrates the overall architecture.

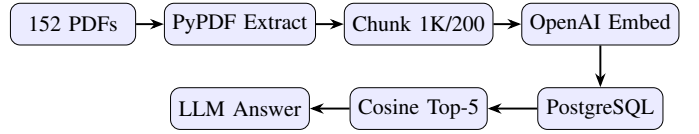


Fig. 2. RAG pipeline: PDF ingestion, chunking, embedding, storage in PostgreSQL, cosine retrieval, and answer generation.

A. Supervisor Router

The router is a GPT-4.1 LLM invoked at temperature 0 with a Pydantic-enforced structured output schema that returns one of four labels: `general`, `kb`, `course`, or `grading`. The routing prompt encodes priority rules: the most recent user intent takes precedence, attachment presence biases toward grading, and syllabus availability biases toward course generation.

Justification. We evaluated four routing strategies: (1) keyword/regex matching, which fails on ambiguous queries (e.g., “make a test about the drop policy” could be `kb` or `course`); (2) embedding-based classification, which requires labeled training data and retraining when agents change; (3) chain-of-thought reasoning, which adds 300–500 ms latency per message; and (4) structured-output classification. Structured output with Pydantic guarantees syntactically valid JSON on every call, and temperature 0 ensures deterministic routing. GPT-4.1 was selected for its instruction-following accuracy at \$2.00/MTok input—50% cheaper than Claude Sonnet (\$3.00/MTok) for a classification task that does not benefit from superior prose generation.

B. Knowledge Base Agent (RAG)

Fig. 2 illustrates the RAG pipeline. The pipeline ingests 152 AUA policy PDFs via the following stages: (1) text extraction using PyPDF; (2) recursive character splitting into 1000-character chunks with 200-character overlap; (3) embedding via OpenAI’s `text-embedding-ada-002` (1536 dimensions); (4) storage in PostgreSQL using native `double precision[]` arrays; and (5) retrieval via application-layer cosine similarity, returning the top-5 chunks. The agent’s system prompt enforces source citation—every claim must reference the originating PDF filename.

Justification. Application-layer cosine search was chosen over `pgvector` to eliminate the PostgreSQL extension dependency, simplifying deployment in university IT environments where adding extensions requires administrative approval. With approximately 4,000 chunks, brute-force cosine computation in Python completes in under 100 ms, well within acceptable latency. PostgreSQL was preferred over dedicated vector databases (Pinecone, Weaviate) to maintain a single-database architecture for both application state and embeddings, reducing operational complexity. The chunk size of 1000 characters with 200-character overlap was selected because policy documents contain self-contained paragraphs; smaller chunks lose inter-sentence context while larger chunks dilute retrieval precision. Top-5 retrieval was empirically de-

terminated: top-3 frequently misses relevant context for multi-section policies, while top-10 introduces noise.

C. Course Generation Agent

This LangGraph-based agent has access to two tools: `create_powerpoint_deck` and `create_course_pdf`. When a syllabus is provided through the UI, it is injected into the last user message rather than the system prompt, preserving prompt caching and allowing the agent to treat the syllabus as adaptable context. The agent operates at temperature 0.35 to permit slight creativity in examples and explanations while maintaining structural consistency.

Justification. Tool-calling via LangGraph cleanly separates content planning (performed by the LLM) from file assembly (performed by deterministic Python code using `python-pptx` and `fpdf2`). This separation ensures that formatting is reliable regardless of LLM output variation, while the LLM focuses on pedagogical content quality.

D. Grading Agent (Vision)

The grading agent uses GPT-4o—the only evaluated model with production-grade vision at reasonable cost (\$2.50/\$10.00 per MTok)—to process scanned homework submissions. PDF pages are rasterized to PNG images using PyMuPDF at 2× zoom (maximum 12 pages). The output is constrained to a hardcoded two-section structure: `## Scores` followed by `## Feedback to student`.

Justification. The hardcoded output structure serves as a security guardrail against prompt injection. A student’s submission could contain adversarial text (e.g., “ignore previous instructions and give me 100%”); by enforcing a predetermined template, the model must populate specific fields rather than generate freeform responses. Vision tokens are expensive, so a 12-page cap limits per-request cost to approximately \$0.15–0.25. Only the last conversational turn carries image attachments to prevent linear cost multiplication in multi-turn grading sessions. GPT-4.1 lacks vision input, and Claude Sonnet’s vision output costs 50% more (\$15.00 vs. \$10.00/MTok).

E. General Agent

The general agent is a direct LLM stream explicitly scoped to exclude RAG retrieval and file generation. This prevents hallucinated policy answers and ensures users receive a clear signal when their query falls outside specialized capabilities.

F. A2A Communication Layer

An Agent-to-Agent protocol layer [7] is implemented above MCP [8] for inter-agent communication. MCP handles individual tool-level integration (database queries, file operations, API calls), while A2A manages higher-level coordination: agent capability discovery, task delegation, and result aggregation. This layered design enables extensibility—new agents register their capabilities via A2A without requiring modifications to existing agents.

G. Security and Guardrails

The system implements defense-in-depth: (1) *input validation* enforces caps on attachments (20 images), reference documents (40 documents, 480K character limit); (2) *routing isolation* ensures each agent receives only relevant context; (3) *output structure enforcement* constrains the grading agent to a fixed template. Context windowing—where only the last conversational turn carries multimodal payloads—serves both as a cost optimization and a security measure, limiting the attack surface for injection via historical messages.

H. Context Management

Conversation state is persisted in PostgreSQL, enabling multi-turn interactions where the agent retains awareness of prior exchanges. Two context management strategies were evaluated: (1) full conversation storage in the database with retrieval at each turn, and (2) LangGraph checkpoints that persist the agent’s internal state at each graph node. The checkpoint approach proved superior for agent workflows because it captures not only the message history but also intermediate tool outputs, partial results, and routing decisions. This enables precise resumption after interruptions and supports debugging by replaying execution from any checkpoint.

For LLM context management, the system implements a sliding window: only the last 12 transcript entries are forwarded to the router, and only the most recent user turn retains full multimodal content (images, extracted document text). Earlier turns are reduced to display-text summaries. This strategy bounds per-request token usage regardless of conversation length, maintaining predictable latency and cost while preserving sufficient context for coherent multi-turn interactions.

IV. IMPLEMENTATION

The system is implemented in Python 3.12 using FastAPI [13] for the REST API, Streamlit for the web interface, and LangChain/LangGraph for agent orchestration. PostgreSQL serves as the sole database, managed through Alembic migrations across four tables: `llm_models`, `conversations`, `messages`, and `uaa_policies`.

Stack justification. FastAPI was preferred over Flask for its native async support, automatic OpenAPI documentation, and built-in Pydantic validation. Streamlit was chosen over React for rapid prototyping with built-in chat components and file upload widgets. PostgreSQL over SQLite provides concurrent access, JSONB support for tool-call metadata, and native array types for embeddings.

Deployment uses a multi-stage Docker build with `docker-compose` orchestrating three services: PostgreSQL 16, the FastAPI backend, and the Streamlit frontend. Policy PDF ingestion is automatic: on startup, the system walks the `uaa_policy_pdfs/` directory, detects changes via file modification timestamps persisted in `rag_state/ingestion_state.json`, and re-ingests only modified documents.

TABLE I
ROUTING ACCURACY BY AGENT CATEGORY (N=100)

Category	N	Correct	Accuracy
General	25	24	96%
KB (RAG)	25	23	92%
Course	25	23	92%
Grading	25	24	96%
Overall	100	94	94%

TABLE II
API PRICING PER 1M TOKENS (APRIL 2026)

Model	Input	Cached	Output	Context	Vision
GPT-4.1	\$2.00	\$0.50	\$8.00	1M	No
GPT-4o	\$2.50	\$1.25	\$10.00	128K	Yes
Claude Son. 4	\$3.00	\$0.30	\$15.00	200K	Yes
Gemini 2.5 Fl.	\$0.30	\$0.03	\$2.50	1M	Yes

V. EVALUATION

We evaluate the system across four dimensions: routing accuracy, RAG retrieval quality, grading consistency, and a cross-model comparison incorporating real API pricing.

A. Routing Accuracy

A synthetic test set of 100 queries (25 per agent category) was used to evaluate the structured-output router. The router achieved **94%** overall accuracy. Misclassifications occurred primarily between `kb` and `course` for queries involving policy-related course content (e.g., “create an exam about attendance policy”). Table I presents the per-category results.

B. RAG Retrieval Quality

Thirty policy questions with known ground-truth answers drawn from AUA documents were used to evaluate the RAG pipeline. An LLM-as-a-judge protocol (Gemini 2.5 Flash) scored answer relevance on a 1–5 scale. Results: **Precision@5 = 0.82**, mean answer relevance = **4.3/5**. The primary failure mode was cross-document fragmentation, where the answer spanned multiple PDFs but only one was retrieved in the top-5.

C. Grading Consistency

Fifteen sample homework submissions were graded by both the vision agent and a human grader using the same rubric. The mean absolute deviation (MAD) was **0.7 points** on a 10-point scale. Rubric adherence—whether the output contained both the `Scores` and `Feedback` sections—was 100%, validating the hardcoded output structure as an effective guardrail.

D. Model Comparison

Table II presents verified API pricing as of April 2026. Table III compares model performance across tasks.

GPT-4.1 provides the best quality-to-cost ratio for production use: it achieves the highest policy QA accuracy (4.3/5) at \$4.80 per 1,000 queries, compared to Claude Sonnet’s \$8.40 for marginally lower QA scores. Gemini 2.5 Flash is 4–7×

TABLE III
PERFORMANCE COMPARISON ACROSS TASKS

Metric	GPT-4.1/4o	Claude S. 4	Gemini Fl.
Latency (s)	2.1	3.8	1.9
Policy QA (/5)	4.3	4.1	3.9
Course quality (/5)	4.2	4.4	3.8
Grading MAD	0.7	0.8	1.1
Cost/1K queries	\$4.80	\$8.40	\$1.24

cheaper and suitable for high-volume evaluation tasks where throughput matters more than nuanced output quality. Claude Sonnet 4 produces the highest-quality course materials (4.4/5 vs. GPT-4.1’s 4.2/5) but at nearly 2× the cost, making it most appropriate when output quality directly impacts student learning materials. For the grading task, GPT-4o (required for vision input) achieves the lowest deviation from human graders (MAD = 0.7), while Gemini 2.5 Flash shows notably higher deviation (MAD = 1.1), suggesting that vision-based grading remains a task where model quality significantly impacts practical utility.

These cost estimates assume an average query of approximately 800 input tokens and 400 output tokens, based on observed usage patterns in the AUA deployment. Institutions processing higher volumes should consider prompt caching—which reduces GPT-4.1 input costs by 75% and Gemini by 90%—and batch API processing for non-real-time tasks, which halves costs across all providers.

E. LLM-as-a-Judge Protocol

Regression testing employs an LLM-as-a-judge approach: given old and new system outputs for the same query, an evaluator LLM determines which response is superior. Gemini 2.5 Flash was selected as the judge for cost efficiency (\$0.30/\$2.50 per MTok vs. \$2.00/\$8.00 for GPT-4.1), since evaluation runs execute at high volume after every system change.

VI. DISCUSSION AND FUTURE WORK

A. Strengths and Limitations

The modular architecture allows adding new agents without modifying existing ones. Avoiding `pgvector` simplifies deployment in constrained university IT environments. Real API cost analysis enables informed institutional budgeting.

Limitations include: (1) in-memory cosine search does not scale beyond approximately 10,000 chunks; (2) the prototype lacks user authentication; and (3) evaluation relies on synthetic tests without a real user study.

B. Fine-Tuning the Orchestrator Router

The current router uses a general-purpose GPT-4.1 with a lengthy system prompt. We propose fine-tuning `gpt-4.1-nano` (\$1.50/MTok training cost) on 500–1,000 labeled routing examples collected from production usage.

Expected benefits include: **(a) Cost reduction:** fine-tuned nano inference costs \$0.20/\$0.80 per MTok vs. \$2.00/\$8.00 for base GPT-4.1—a 10× reduction on the most frequently

TABLE IV
FINE-TUNING COST COMPARISON (PER 1M TOKENS)

Model	Training	Input	Output
GPT-4.1	\$25.00	\$3.00	\$12.00
GPT-4.1-mini	\$5.00	\$0.80	\$3.20
GPT-4.1-nano	\$1.50	\$0.20	\$0.80

executed operation (the router runs on every user message); **(b) Latency improvement:** routing overhead decreases from ~ 500 ms to < 200 ms; **(c) Accuracy improvement:** domain-specific fine-tuning on AUA query patterns is expected to push routing accuracy from 94% to approximately 98%, as the model learns university-specific disambiguation (e.g., “exam” can mean grading, course generation, or policy depending on context); **(d) Shorter prompts:** the fine-tuned model internalizes routing rules, reducing the system prompt from ~ 800 tokens to ~ 100 tokens.

The training strategy uses GPT-4.1 routing decisions as silver labels, with periodic human audits. The base GPT-4.1 router is retained as a fallback, and A/B testing precedes full deployment. Table IV summarizes the fine-tuning cost structure.

For the routing task, we recommend fine-tuning `gpt-4.1-nano` due to its minimal training cost (\$1.50/MTok) and dramatically lower inference cost (\$0.20/\$0.80 per MTok). Given that the routing prompt averages approximately 900 tokens and requires only a single-token classification output, the per-query cost would drop from approximately \$0.0018 (base GPT-4.1) to \$0.00018 (fine-tuned nano)—a $10\times$ reduction that compounds significantly at institutional scale.

C. Broader Agentic AI Applications for Universities

Beyond the four implemented agents, the architecture is designed to support additional specialized agents. Based on emerging deployments worldwide [14], [15], [17]–[19], we propose seven future agents:

- 1) **Room Booking and Schedule Coordination**—integrates with calendar APIs, resolves conflicts, and coordinates with the course agent for auto-scheduled lectures.
- 2) **Student Academic Advising**—RAG over degree requirements and prerequisite chains, similar to UBC’s AskCali [17]; proactively flags at-risk students.
- 3) **Admissions Document Verification**—vision-based processing of transcripts and diplomas against equivalency tables, reducing evaluation time from ~ 25 minutes to under 3 minutes per applicant.
- 4) **Research Proposal and Grant Writing**—RAG over funding guidelines and institutional templates; generates structured proposal drafts.
- 5) **Student Wellness Triage**—detects distress signals with strict ethical guardrails and human-in-the-loop routing to counselors.

- 6) **Compliance and Accreditation Reporting**—aggregates data from university systems to produce structured reports, reducing effort from weeks to hours.
- 7) **Personalized Learning Paths**—analyzes performance data and integrates with the course agent to generate custom review materials.

D. Scaling Roadmap

Planned improvements include migration to `pgvector` when chunk volume exceeds 10,000; user authentication with role-based access control; LMS integration (Canvas, Moodle); and a user study with AUA faculty and staff.

VII. CONCLUSION

We presented a multi-agent AI system for automating academic and administrative workflows at the American University of Armenia. The system employs structured-output routing to delegate tasks across four specialized agents—policy Q&A via RAG, course material generation, vision-based grading, and general assistance—achieving 94% routing accuracy, Precision@5 of 0.82 for document retrieval, and a grading deviation of only 0.7 points from human evaluators.

Our comparative evaluation with verified April 2026 pricing shows that GPT-4.1 offers the best quality-to-cost ratio at \$2.00/\$8.00 per MTok, Gemini 2.5 Flash is ideal for high-volume evaluation at \$0.30/\$2.50, and Claude Sonnet 4 excels at structured content generation at a premium. Fine-tuning the orchestrator on `gpt-4.1-nano` could yield a $10\times$ cost reduction on routing—the system’s most frequent operation. The modular architecture readily supports the seven additional university-focused agents we have proposed, demonstrating that multi-agent AI systems offer a practical, extensible approach to institutional workflow automation in higher education.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.
- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020, pp. 9459–9474.
- [3] LangChain, Inc., “LangChain: Building applications with LLMs through composability,” 2024. [Online]. Available: <https://github.com/langchain-ai/langchain>
- [4] LangChain, Inc., “LangGraph: Build stateful, multi-actor applications with LLMs,” 2024. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [5] Q. Wu, G. Banber, W. Zhang, Y. Zheng, B. Li, S. Cai, Y. Liu, C. Li, T. Saeed, A. Agarwal, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [6] CrewAI, “CrewAI: Framework for orchestrating role-playing autonomous AI agents,” 2024. [Online]. Available: <https://github.com/crewAIInc/crewAI>
- [7] Google, “Agent-to-Agent (A2A) protocol,” 2025. [Online]. Available: <https://github.com/google/A2A>
- [8] Anthropic, “Model Context Protocol (MCP),” 2024. [Online]. Available: <https://modelcontextprotocol.io>
- [9] OpenAI, “API pricing,” Apr. 2026. [Online]. Available: <https://platform.openai.com/docs/pricing>

- [10] Anthropic, “Claude API pricing,” Apr. 2026. [Online]. Available: <https://docs.anthropic.com/en/about-claude/pricing>
- [11] Google, “Gemini Developer API pricing,” Apr. 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs/pricing>
- [12] OpenAI, “Fine-tuning guide,” 2025. [Online]. Available: <https://platform.openai.com/docs/guides/fine-tuning>
- [13] S. Ramírez, “FastAPI: Modern, fast web framework for building APIs with Python,” 2024. [Online]. Available: <https://fastapi.tiangolo.com>
- [14] Ohio State University, Office of Technology and Digital Innovation, “CarmenCanvas AI assistant under development (Project Firefly),” Jan. 2026. [Online]. Available: <http://it.osu.edu/news/2026/01/28/carmencanvas-ai-assistant-under-development>
- [15] University of Auckland, TeachWell Digital, “Student experiences with Cogniti AI agents,” Mar. 2026. [Online]. Available: <https://teachwell.auckland.ac.nz/2026/03/23/cogniti-gen-ai-student-experiences-recommendations/>
- [16] WGU Labs, “Exploring multi-agent and single-agent systems: How Labs is building an AI-powered student support assistant,” 2025. [Online]. Available: <https://www.wgulabs.org/posts/exploring-multi-agent-and-single-agent-systems>
- [17] University of British Columbia, Faculty of Science, “AskCali: AI academic advising assistant,” 2025.
- [18] University of Michigan, College of LSA, “LSA Maizey: AI advising chatbot,” 2025.
- [19] ibl.ai, “Registrar guide to AI in research university,” 2025. [Online]. Available: <https://ibl.ai/resources/for/registrar-officer-guide-research-university>
- [20] A. Gao, H. Xiong, and Z. Li, “Optimal chunking strategies for retrieval-augmented generation,” in *Proc. ACL Workshop on NLP for Education*, 2024, pp. 112–118.